

Checkstyle, uma ferramenta que verifica padrões e qualidades de códigos, foi utilizada para fazer a checagem das classes no nosso código onde nas imagens como exemplo podemos ver o relatório entregue das classes.

demoVersion control -

javaSolicitacao.javaSolicitacaoRepository.javaSolicitacaoDTO.javaAtualizarStatusDTO.javaServicoSolicitacao.javaDemoApplication.javaPOM.xmlcheckstyle.xml

Project -

CheckStyle

Scan

Rules:Sun Checks

Checkstyle found 71 items in 1 file(s)
Solicitacao.java - 71 item(s)

- O arquivo package-info.java está faltando. (1:0) [JavadocPackage] <Sun Checks>
- A forma de importação "*" deveria ser evitada - jakarta.persistence.* (5:27) [AvoidStarImport] <Sun Checks>
- Falta o comentário Javadoc. (33:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (37:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (40:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (43:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (44:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (45:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (46:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (48:5) [JavadocVariable] <Sun Checks>
- Falta o comentário Javadoc. (55:5) [MissingJavadocMethod] <Sun Checks>
- " " não está precedido por espaço em branco. (55:25) [WhitespaceAround] <Sun Checks>
- A classe "Solicitacao" parece ter sido projetada para extensão (permite subclasses), mas o método "podeMudarPara" não tem um javadoc que explica como fazer isso de forma segura. Se a classe não é projetada para extensão, considere fazer a classe "Solicitacao" ser final. (58:5) [MissingJavadocMethod] <Sun Checks>
- Falta o comentário Javadoc. (58:5) [MissingJavadocMethod] <Sun Checks>
- O parâmetro novoStatus deve ser final. (58:34) [FinalParameters] <Sun Checks>
- A estrutura "if" deve utilizar "()"s. (60:9) [NeedBraces] <Sun Checks>
- "if" não está seguido de espaço em branco. (60:9) [WhitespaceAround] <Sun Checks>
- A estrutura "if" deve utilizar "()"s. (63:9) [NeedBraces] <Sun Checks>
- "if" não está seguido de espaço em branco. (63:9) [WhitespaceAround] <Sun Checks>
- A estrutura "if" deve utilizar "()"s. (66:9) [NeedBraces] <Sun Checks>
- "if" não está seguido de espaço em branco. (66:9) [WhitespaceAround] <Sun Checks>
- A estrutura "if" deve utilizar "()"s. (69:9) [NeedBraces] <Sun Checks>
- "if" não está seguido de espaço em branco. (69:9) [WhitespaceAround] <Sun Checks>
- A classe "Solicitacao" parece ter sido projetada para extensão (permite subclasses), mas o método "getProtocolo" não tem um javadoc que explica como fazer isso de forma segura. Se a classe não é projetada para extensão, considere fazer a classe "Solicitacao" ser final. (75:5) [MissingJavadocMethod] <Sun Checks>
- Falta o comentário Javadoc. (75:5) [MissingJavadocMethod] <Sun Checks>
- A classe "Solicitacao" parece ter sido projetada para extensão (permite subclasses), mas o método "setProtocolo" não tem um javadoc que explica como fazer isso de forma segura. Se a classe não é projetada para extensão, considere fazer a classe "Solicitacao" ser final. (79:5) [MissingJavadocMethod] <Sun Checks>
- Falta o comentário Javadoc. (79:5) [MissingJavadocMethod] <Sun Checks>
- O parâmetro protocolo deve ser final. (79:30) [FinalParameters] <Sun Checks>
- O protocolo "esconde" um campo. (79:38) [HiddenField] <Sun Checks>
- A classe "Solicitacao" parece ter sido projetada para extensão (permite subclasses), mas o método "getStatus" não tem um javadoc que explica como fazer isso de forma segura. Se a classe não é projetada para extensão, considere fazer a classe "Solicitacao" ser final. (83:5) [MissingJavadocMethod] <Sun Checks>
- Falta o comentário Javadoc. (83:5) [MissingJavadocMethod] <Sun Checks>
- A classe "Solicitacao" parece ter sido projetada para extensão (permite subclasses), mas o método "setStatus" não tem um javadoc que explica como fazer isso de forma segura. Se a classe não é projetada para extensão, considere fazer a classe "Solicitacao" ser final. (87:5) [MissingJavadocMethod] <Sun Checks>
- Falta o comentário Javadoc. (87:5) [MissingJavadocMethod] <Sun Checks>

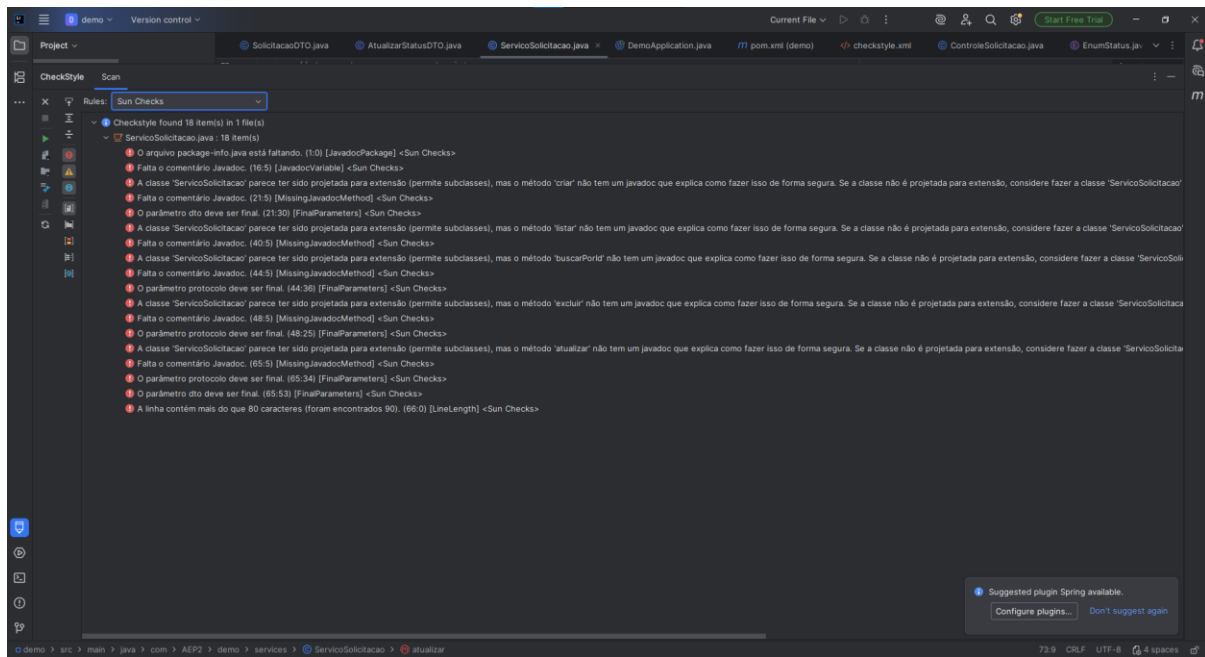
Tais Problemas apareceram especificamente para apontar problemas na estética do código relacionada à ausência de comentários Javadoc, utilização de estruturas condicionais sem chaves, problemas de espaçamento e padronização de código.

Apesar da quantidade de inconsistências nenhum dos problemas apontadas no relatório impactam diretamente na execução do sistema, mas sim na manutenção de software, já que a ausência de documentação pode causar muitas dificuldades na interpretação do código por outros programadores e os problemas de formatação reduzem a legibilidade e podem acrescentar muito tempo a mais necessário para futuras correções e evoluções.

A melhorar:

Portanto melhorias que podem ser aplicadas seria, adicionar documentação Javadoc nos métodos e atributos principais, padronizar as escritas das estruturas condicionais e seguir as métricas recomendadas na utilização do Java para programar.

Ex2: Classe ServicoSolicitacao



Problemas encontrados:

Alguns dos principais problemas encontrados no Service estão relacionados novamente à ausência de comentários Javadoc nos métodos e atributos, parâmetros que poderiam ser declarados como final para não ser herdados sem permissão por outras classes e há muitas linhas de códigos que ultrapassa o limite de caracteres recomendado pela ferramenta que é 80 caracteres.

Impactos:

Apesar das inconsistências apontadas, novamente nenhuma delas afeta o funcionamento do sistema impacto está na manutenção do software, em que assim como o primeiro exemplo ausência de documentação e problemas de formatação reduzem a legibilidade e podem acrescentar muito tempo a mais necessário para futuras correções e evoluções.

A melhorar:

Adicionar comentários Javadoc nos principais métodos e atributos ajudaria bastante, utilizar final em certos parâmetros torna as classes não herdáveis sem permissão e por fim adequar linhas muito extensas, marcadas por atingir 80 linhas para seguir as métricas recomendadas na utilização do Java para programar.